M02 — Asymptotics & the Analysis Toolkit

Sources: CLRS Ch. 3 (Characterizing Running Times) + Appendix A (Summations) · Skiena Ch. 2 (Algorithm Analysis)

Big Idea

Exact running-time functions are unusable: they are full of bumps, depend on coding trivia, and look like $T(n) = 12754n^2 + 4353n + 834 \lg^2 n + 13546$. Asymptotic notation throws away everything except the **rate of growth** — the one property that survives changing language, compiler, and machine, and the one that decides which algorithm wins on large inputs. The five notations $O, \Omega, \Theta, o, \omega$ are exactly the asymptotic analogues of $\leq, \geq, =, <, >$, and the entire skill is *going back to the definition* whenever intuition wobbles. Two mechanical toolkits do the actual work: the **dominance hierarchy** ($n! \gg c^n \gg n^3 \gg n^2 \gg n \log n \gg n \gg \sqrt{n} \gg \log^2 n \gg \log n \gg \log\log n \gg \alpha(n) \gg 1$) and a handful of **summation closed forms** (arithmetic → $\Theta(n^2)$, geometric → dominated by its largest term, harmonic → $\Theta(\log n)$). Months later, remember two things: *nested loops multiply, sequential blocks take the max*, and *whenever something is repeatedly halved or doubled, a logarithm appears.*

What You Should Be Able To Do After This Chapter

- State the formal set definitions of O, Ω, Θ, o, ω and prove membership by exhibiting c and n_0.
- Prove a *non*-membership (e.g. $n^3 - 100n^2 \notin O(n^2)$) by deriving a contradiction from the definition.
- Say precisely why "algorithm A runs in $\Theta(n^2)$" can be wrong while "A's *worst-case* running time is $\Theta(n^2)$" is right.
- Rank any list of standard functions by growth, and place unfamiliar ones (`n^{1/lg n}`, `lg*n`, `(lg n)!`) into the hierarchy.
- Analyze nested loops by turning them into nested summations and evaluating from the inside out.
- Prove a Θ bound by constructing a worst-case instance that forces Ω, not just

hand-waving the `O`.

- Recall the closed forms for arithmetic, geometric, harmonic, telescoping series and know when a geometric series is "free".
- Explain why the base of a logarithm never matters inside asymptotic notation, and where logs come from (halving, tree height, bit width, `lg n!`).
- Use limits (`lim f/g`) to settle dominance when the hierarchy doesn't obviously apply.

---

1. Why we need asymptotic notation

Problem

Best-, worst-, and average-case complexity are perfectly well-defined numerical functions of `n` [Skiena §2.1.1, p.33]. They are also unusable directly.

Skiena's two reasons [§2.2, p.34]

1. Too many bumps. Binary search runs slightly faster when `n = 2^k − 1`, because the partitions divide evenly. The exact function is jagged with small local wiggles that carry no information.

2. Too much detail to specify precisely. Counting exact RAM instructions requires the algorithm pinned down to a complete program, and the answer then depends on whether you wrote a `switch` or nested `if`s. `T(n) = 12754n² + 4353n + 834 lg²n + 13546` is a lot of work for no more information than *"the time grows quadratically."*

The abstraction, in three steps [CLRS §3.1, p.50]

Starting from insertion sort's exact worst case:

```
(c₅/2 + c₆/2 + c₇/2)·n² + (c₁ + c₂ + c₄ + c₅/2 − c₆/2 − c₇/2 +
c₈)·n − (c₂ + c₄ + c₅ + c₈)
```

1. Discard the lower-order terms.
2. Discard the coefficient of the leading term.
3. Wrap what's left in `Θ`: **`Θ(n²)`**.

Why discarding constants is legitimate: if a C implementation runs twice as fast as a Java one of *the same algorithm*, that factor of two says nothing about the algorithm. Constant factors describe the implementation; the growth rate describes the algorithm. [Skiena §2.2, p.34]

CLRS's framing: we study **asymptotic efficiency** — how running time increases *in the limit*, as input size grows without bound. And the payoff: *"Usually, an algorithm that is asymptotically more efficient is the best choice for all but very small inputs."*

Important scoping note [CLRS §3.1, p.50]: asymptotic notation characterizes **functions in general**. Running time is just the function we care about most. It applies equally to space usage, number of comparisons, or functions with nothing to do with algorithms at all.

---

2. The five notations — formal definitions

Unified Understanding

All five are **sets of functions**, defined by a bound that holds *for all sufficiently large* n.

NOTATION	DEFINITION (CLRS §3.2)	READS AS	REAL-NUMBER ANALOGUE
$O(g(n))$	$\{f : \exists\ c > 0, n_0 > 0 \text{ s.t. } 0 \le f(n) \le c \cdot g(n)\ \forall\ n \ge n_0\}$	grows **no faster than**	$a \le b$
$\Omega(g(n))$	$\{f : \exists\ c > 0, n_0 > 0 \text{ s.t. } 0 \le c \cdot g(n) \le f(n)\ \forall\ n \ge n_0\}$	grows **at least as fast as**	$a \ge b$
$\Theta(g(n))$	$\{f : \exists\ c_1, c_2 > 0, n_0 > 0 \text{ s.t. } 0 \le c_1 g(n) \le f(n) \le c_2 g(n)\ \forall\ n \ge n_0\}$	grows **exactly as fast as** (to within constants)	$a = b$
$o(g(n))$	$\{f : \forall\ c > 0, \exists\ n_0 > 0 \text{ s.t. } 0 \le f(n) < c \cdot g(n)\ \forall\ n \ge n_0\}$	grows **strictly slower than**	$a < b$
$\omega(g(n))$	$\{f : \forall\ c > 0, \exists\ n_0 > 0 \text{ s.t. } 0 \le c \cdot g(n) < f(n)\ \forall\ n \ge n_0\}$	grows **strictly faster than**	$a > b$

The single structural difference that matters: in O, the bound holds **for some** constant c. In o, it holds **for all** constants c. That quantifier flip is what makes o a strict bound.

Limit characterizations (when the limit exists):

```
f = o(g)   ⟺   lim_{n→∞} f(n)/g(n) = 0
f = ω(g)   ⟺   lim_{n→∞} f(n)/g(n) = ∞
f = Θ(g)   ⟺   lim_{n→∞} f(n)/g(n) = c,   0 < c < ∞
```

Asymptotic nonnegativity. Every function inside these notations must be **asymptotically nonnegative** — nonnegative for all sufficiently large `n`. Otherwise `O(g(n))` is the empty set. Both books assume this throughout. [CLRS §3.2, p.55]

Theorem 3.1 — the one theorem you actually use

> `f(n) = Θ(g(n))` ***if and only if*** `f(n) = O(g(n))` *and* `f(n) = Ω(g(n))`.

This is the standard route to a tight bound: prove the upper bound, prove the lower bound separately, conclude `Θ`.

The "=" abuse, and why it is fine

Formally `O(g(n))` is a set, so we should write `f(n) ∈ O(g(n))`. Both books write `=`. CLRS gives it a precise meaning:

- **Alone on the RHS:** `4n² + 100n + 500 = O(n²)` means set membership.
- **Inside a formula:** the notation stands for **some anonymous function we don't care to name**. `2n² + 3n + 1 = 2n² + Θ(n)` means `2n² + 3n + 1 = 2n² + f(n)` for some `f ∈ Θ(n)`.
- **Number of anonymous functions = number of appearances.** `Σᵢ₌₁ⁿ O(i)` has **one** anonymous function (of `i`), not `n` of them. It is *not* `O(1) + O(2) + … + O(n)`, which has no clean meaning.
- **On the LHS:** `2n² + Θ(n) = Θ(n²)` means *no matter how the anonymous functions on the left are chosen, there is a way to choose those on the right to make it valid.* The RHS is always a **coarser** level of detail.

Skiena's blunter version: read `=` as "**is one of the functions that are**". `n² = O(n³)` reads "`n²` is one of the functions that are `O(n³)`". [Skiena §2.2, p.36]

> *In mathematics, it's okay — and often desirable — to abuse a notation, as long as we don't misuse it. [CLRS §3.2, p.60]*

Worked membership proofs [CLRS §3.2, p.55]

$4n^2 + 100n + 500 = O(n^2)$. Need c, n_0 with $4n^2 + 100n + 500 \le cn^2$ for $n \ge n_0$. Divide by n^2: $4 + 100/n + 500/n^2 \le c$. Many choices work:

n_0	WORKS WITH c =
1	604
10	19
100	5.05

$n^3 - 100n^2 \notin O(n^2)$ — note the large *negative* coefficient does not save it. If it were, there would be c, n_0 with $n^3 - 100n^2 \le cn^2$. Divide by n^2: $n - 100 \le c$. This fails for every $n > c + 100$, whatever c is. ∎

$n^2/100 - 100n - 500 = \Omega(n^2)$ — note the tiny positive coefficient does not hurt it. Divide by n^2: $1/100 - 100/n - 500/n^2 \ge c$. Any $n_0 \ge 10005$ gives a positive c (e.g. $c = 2.49 \times 10^{-9}$). Tiny, but positive — which is all the definition asks. Larger n_0 lets c approach $1/100$.

The lesson from these three: the constants are allowed to be absurd. Do not reject a bound because the constant is ugly.

Skiena's method — always go back to the definition

> *Designing novel algorithms requires cleverness and inspiration. However, applying the Big Oh notation is best done by **swallowing any creative instincts you may have**. All Big Oh problems can be correctly solved by going back to the definition and working with that. [Skiena §2.2, p.37]*

Is $2^{n+1} = \Theta(2^n)$?

- O? $2^{n+1} = 2 \cdot 2^n \le c \cdot 2^n$ for any $c \ge 2$. ✓
- Ω? $2^{n+1} \ge c \cdot 2^n$ for any $0 < c \le 2$. ✓
- Together: Θ. ✓

Contrast $2^{2n} = (2^n)^2$, which is **not** $O(2^n)$ — the ratio is 2^n, unbounded. [CLRS Ex. 3.2-3]

Is $(x + y)^2 = O(x^2 + y^2)$? Expand: $x^2 + 2xy + y^2$. Without the cross term, $c = 1$ works. To bound $2xy$: if $x \le y$ then $2xy \le 2y^2 \le 2(x^2 + y^2)$; if $x \ge y$ then $2xy \le 2x^2 \le 2(x^2 + y^2)$. Either way $(x+y)^2 \le 3(x^2 + y^2)$. ✓ [Skiena §2.2, p.37]

---

3. Using the notation correctly (the interview trap)

CLRS §3.2, p.56 — the precision rules

STATEMENT	CORRECT?	WHY
Insertion sort's **worst-case** running time is $O(n^2)$	✓	true, but not tight
Insertion sort's **worst-case** running time is $\Omega(n^2)$	✓	true, but not tight
Insertion sort's **worst-case** running time is $\Theta(n^2)$	✓ **preferred**	tight
Insertion sort's **best-case** running time is $\Theta(n)$	✓ **preferred**	tight
Insertion sort's running time is $O(n^2)$	✓	blanket statement over all cases; O allows faster cases
Insertion sort's running time is $\Theta(n^2)$	✗	**overstatement** — the blanket claim is false since the best case is $\Theta(n)$
Insertion sort's running time is $\Theta(n)$	✗	worst case is not $\Theta(n)$
Insertion sort's running time is $\Omega(n)$	✓	true in all cases
Merge sort's running time is $\Theta(n \log n)$	✓	no qualifier needed — same in all cases

Read that table twice. Dropping "worst-case" turns a Θ claim into a claim about *every* case, which is usually false.

The most common conflation

> *People occasionally conflate `O`-notation with `Θ`-notation by mistakenly using `O` to indicate an asymptotically tight bound. They say things like "an `O(n lg n)`-time algorithm runs faster than an `O(n²)`-time algorithm." **Maybe it does, maybe it doesn't.** Since `O` denotes only an asymptotic upper bound, that so-called `O(n²)`-time algorithm might actually run in `Θ(n)` time. [CLRS §3.2, p.57]*

"The running time of algorithm A is at least `O(n²)`" is meaningless [CLRS Ex. 3.2-2]: `O` is already an upper bound, so "at least an upper bound" says nothing — every running time is at least `O(n²)` in the sense that some upper bound `O(n²)` may or may not hold. If you mean a lower bound, say `Ω`.

Choose the simplest, most precise bound

If a running time is `3n² + 20n` in all cases, `Θ(n²)` is the right statement. `O(n³)` is correct but less precise. `Θ(3n² + 20n)` is correct but "introduces complexity that obscures the order of growth."

---

4. Proving a `Θ` bound: the `Ω` half is the work

Problem

Getting the `O` is usually a mechanical loop count. Getting the `Ω` requires **constructing an input that forces the algorithm to be slow**.

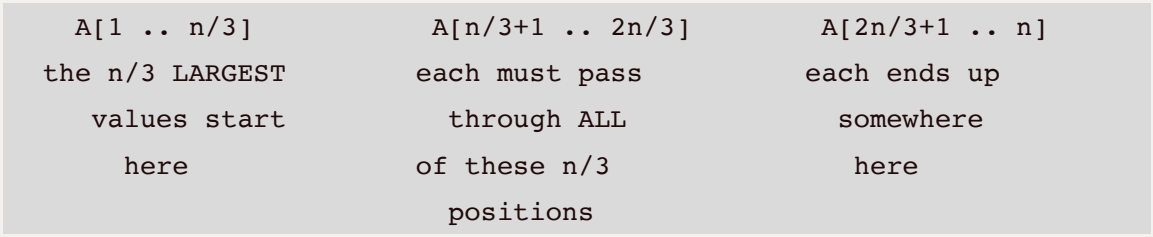
> *Generally speaking, turning a Big Oh worst-case analysis into a Big Θ involves **identifying a bad input instance that forces the algorithm to perform as poorly as possible**. [Skiena §2.5.1, p.42]*

What `Ω` on a worst case actually asserts

> *By saying that the worst-case running time of an algorithm is `Ω(n²)`, we mean that **for every input size `n` above a certain threshold, there is at least one input of size `n`** for which the algorithm takes at least `cn²` time. It does **not** mean the algorithm takes at least `cn²` time for all inputs. [CLRS §3.1, p.52]*

Worked example — insertion sort's `Ω(n²)` (CLRS's thirds argument)

Assume `n` divisible by 3. Split the array into three blocks of `n/3`.



For a value to end up `k` positions right of where it started, line 6 (the shift) must execute `k` times. If the `n/3` largest values start in the first third, every one of them must end in the last third, hence must pass through **every one of the `n/3` middle positions**, one position at a time. That is at least `(n/3)·(n/3) = n²/9` shifts.

Combined with the `O(n²)` upper bound (`n−1` outer iterations × at most `n−1` inner iterations of constant work), insertion sort's **worst case is `Θ(n²)`** — without evaluating a single summation. [CLRS §3.1, p.52]

Skiena's shorter version of the same argument: reverse-sorted input; the last `n/2` elements each slide over at least `n/2` positions, giving `(n/2)² = Ω(n²)`. [Skiena §2.5.2, p.43]

Selection sort — the algorithm with no bad case

```cpp
void selectionSort(vector<int>& s) {
    const int n = static_cast<int>(s.size());
    for (int i = 0; i < n; ++i) {
        int mn = i;
        for (int j = i + 1; j < n; ++j)
            if (s[j] < s[mn]) mn = j;
        swap(s[i], s[mn]);
    }
}
```

Number of `if` executions:

```
T(n) = Σ_{i=0}^{n-1} Σ_{j=i+1}^{n-1} 1 = Σ_{i=0}^{n-1} (n − i − 1)
     = (n-1) + (n-2) + … + 2 + 1 = n(n-1)/2
```

→ **C++ implementation:** A2 Selection sort's exact comparison count

Upper bound: `n` terms each at most `n-1` → `T(n) ≤ n(n-1) = O(n²)`.
Lower bound: the first `n/2` terms are each `> n/2`, the rest ⩾ 0 → `T(n) ≥ (n/2)(n/2) = Ω(n²)`.

But selection sort is special: **it takes exactly the same time on all `n!` inputs.** `T(n) = n(n-1)/2` for every input, so `T(n) = θ(n²)` with no case analysis at all. [Skiena §2.5.1, p.42]

Worked example — naive string matching, in full

```
// Returns index of first occurrence of p in t, or -1.
int findMatch(const string& p, const string& t) {
    const int m = static_cast<int>(p.size());
    const int n = static_cast<int>(t.size());
    for (int i = 0; i + m <= n; ++i) {
        int j = 0;
        while (j < m && t[i + j] == p[j]) ++j;
        if (j == m) return i;
    }
    return -1;
}
```

Skiena's simplification chain [§2.5.3, p.44] — worth following because it is the *style* of asymptotic algebra:

```
inner while ≤ m iterations; outer for ≤ n − m iterations; plus 2
statements
  →  O((n − m)(m + 2))
plus strlen on both strings:  O(n + m + (n − m)(m + 2))
m + 2 = θ(m)                  →  O(n + m + (n − m)·m)
expand                        →  O(n + m + nm − m²)
n ≥ m in any interesting case, so n + m ≤ 2n = θ(n)
                              →  O(n + nm − m²)
m ≥ 1, so n ≤ nm, hence n + nm = θ(nm)
                              →  O(nm − m²)
−m² is negative and (since mn ≥ m²) cannot cancel the leading term;
dropping a negative term keeps an upper bound valid
                              →  O(nm)
```

→ **C++ implementation:** A3 Naive string matching, and its Ω(nm) instance

The `Ω(nm)` instance: `t = "aaaa…a"` (`n` a's), `p = "aaa…ab"` (`m−1` a's then `b`). At each of the `n − m + 1` alignments the inner loop matches `m−1` characters and fails on the last.

```
(n − m + 1)·m = nm − m² + m = Ω(nm)
```

Hence **`Θ(nm)`**. Faster algorithms exist — Rabin–Karp (expected linear) and KMP (worst-case linear) in M18 *(planned)*.

The "round it up" heuristic, and its limit

> *A basic rule of thumb in Big Oh analysis is that worst-case running time follows from **multiplying the largest number of times each nested loop can iterate**. … This crude "round it up" analysis always does the job, in that the Big Oh bound you get will always be **correct**. Occasionally it might be too pessimistic, meaning the actual worst-case time might be of a lower order. [Skiena §2.5.2, p.43]*

Where it over-estimates: amortized structures (M09), union-find (M10), and any loop whose total work is bounded globally rather than per-iteration.

---

5. The dominance hierarchy

Definition

`g` **dominates** `f` (written `g ≫ f`) when `lim_{n→∞} f(n)/g(n) = 0` — i.e. `f = o(g)`. [Skiena §2.10.2, p.58]

The full ordering (Skiena's, extended)

```
n!  ≫  cⁿ  ≫  n³  ≫  n²  ≫  n^{1+ε}  ≫  n log n  ≫  n  ≫  √n
    ≫  log²n  ≫  log n  ≫  log n / log log n  ≫  log log n  ≫
α(n)  ≫  1
```

→ **C++ implementation:** A6 The dominance hierarchy, measured

The bread-and-butter classes [Skiena §2.3.1, p.38]

CLASS	FORM	WHERE IT COMES FROM
Constant	`1`	adding two numbers; `min(n, 100)`
Logarithmic	`log n`	binary search — repeated halving
Linear	`n`	one pass over `n` items
Superlinear	`n log n`	quicksort, mergesort
Quadratic	`n²`	all **pairs** of `n` items — insertion sort, selection sort
Cubic	`n³`	all **triples** — some DP, naive matrix multiply
Exponential	`cⁿ`	all **subsets** of `n` items
Factorial	`n!`	all **permutations** / orderings of `n` items

This mapping is the fastest complexity check there is: if your algorithm enumerates pairs it is `n²`; subsets, `2ⁿ`; orderings, `n!`.

The esoteric functions [Skiena §2.10.1, p.57]

FUNCTION	WHERE IT ARISES
`α(n)` — inverse Ackermann	union-find with union by rank + path compression (M10). "Geek talk for the slowest growing complexity function." `α(n) < 5` for any `n` writable in this universe.
`log log n`	binary search on a sorted array of only `lg n` items
`log n / log log n`	height of an `n`-leaf tree of degree `d = log n`: `n = (log n)^h ⟹ h = log n / log log n`
`log² n`	binary search on `n` items each an integer up to `n²` — `lg n` probes × `2 lg n` bits each. Also intricate nested data structures (a tree whose every node holds another tree, ordered on a different key).
`√n`	`d`-dimensional grids: a `√n × √n` square has area `n`; `n^{1/3}` cubed has volume `n`
`n^{1+ε}`	an algorithm running in `2^c · n^{1+1/c}` where you pick `c` — the exponent improves with larger `c`, but `2^c` eventually dominates, so we report `O(n^{1+ε})` and "leave the best value of ε to the beholder"
`lg* n` — iterated log	`min{i ≥ 0 : lg^{(i)} n ≤ 1}`. `lg*2 = 1, lg*4 = 2, lg*16 = 3, lg*65536 = 4, lg*(2^65536) = 5`. Since the observable universe has ~`10⁸⁰` atoms and `2^65536 ≈ 10^19728`, **you will never see `lg* n > 5`.** [CLRS §3.3, p.68]

The concrete running-time table [Skiena Fig. 2.4, p.38]

At 1 nanosecond per operation:

N	LG N	N	N LG N	N^2	2^n	N!
10	0.003 µs	0.01 µs	0.033 µs	0.1 µs	1 µs	3.63 ms
20	0.004 µs	0.02 µs	0.086 µs	0.4 µs	1 ms	77.1 years
30	0.005 µs	0.03 µs	0.147 µs	0.9 µs	1 sec	8.4×10^{15} yrs
40	0.005 µs	0.04 µs	0.213 µs	1.6 µs	18.3 min	—
50	0.006 µs	0.05 µs	0.282 µs	2.5 µs	13 days	—
100	0.007 µs	0.1 µs	0.644 µs	10 µs	4×10^{13} yrs	—
10^3	0.010 µs	1.00 µs	9.966 µs	1 ms	—	—
10^4	0.013 µs	10 µs	130 µs	100 ms	—	—
10^5	0.017 µs	0.10 ms	1.67 ms	10 sec	—	—
10^6	0.020 µs	1 ms	19.93 ms	16.7 min	—	—
10^7	0.023 µs	0.01 sec	0.23 sec	1.16 days	—	—
10^8	0.027 µs	0.10 sec	2.66 sec	115.7 days	—	—
10^9	0.030 µs	1 sec	29.90 sec	31.7 years	—	—

Conclusions to memorize:

- All of them are the same at `n = 10`.
- `n!` is useless for `n ≥ 20`.
- `2ⁿ` is impractical past `n ≈ 40`.
- `n²` is usable to `n ≈ 10⁴`, hopeless past `10⁶`.
- `n` and `n lg n` are practical at a **billion** items.
- `lg n` "hardly sweats for any imaginable value of `n`."

Polynomials vs exponentials vs logs (CLRS's three facts)

```
n^b = o(aⁿ)      for all real a > 1, b        (3.13)  — any
exponential beats any polynomial
lg^b n = o(n^a)  for all real a > 0, b        (3.24)  — any
polynomial beats any polylog
n! = o(nⁿ),  n! = ω(2ⁿ),  lg(n!) = θ(n lg n)  (3.26–3.28)
```

Stirling's approximation [CLRS eq. 3.25]:

```
n! = √(2πn) · (n/e)ⁿ · (1 + Θ(1/n))
```

This is what proves `lg(n!) = Θ(n lg n)` — the bound behind the comparison-sorting lower bound in M05.

Proving dominance by limits [Skiena §2.10.2]

COMPARISON	LIMIT	CONCLUSION
`2n²` vs `n²`	`n²/2n² = 1/2 ≠ 0`	neither dominates — both `Θ(n²)`
`n³` vs `n²`	`n²/n³ = 1/n → 0`	`n³ ≫ n²`
`n^a` vs `n^b`, `a > b`	`n^{b−a} → 0`	higher degree dominates (`n^{1.2} ≫ n^{1.1999999}`)
`3ⁿ` vs `2ⁿ`	`(2/3)ⁿ → 0`	larger base dominates
`n^ε` vs `lg n`	`lg n / 2^{ε lg n} → 0`	any polynomial dominates any log

Trichotomy fails

Real numbers satisfy trichotomy: exactly one of `a < b`, `a = b`, `a > b`. **Asymptotic comparison does not.** For `f(n) = n` and `g(n) = n^{1+sin n}`, the exponent oscillates over `[0, 2]`, so neither `f = O(g)` nor `f = Ω(g)` holds. [CLRS §3.2, p.62]

Which properties **do** carry over: transitivity (all five), reflexivity (`O, Ω, Θ`), symmetry (`Θ` only), transpose symmetry (`f = O(g) ⟺ g = Ω(f)`; `f = o(g) ⟺ g = ω(f)`).

---

6. Algebra of asymptotic notation

The two rules that do 90% of the work [Skiena §2.4]

Addition — the dominant term wins.

```
f(n) + g(n) = Θ(max(f(n), g(n)))
```

So $n^3 + n^2 + n + 1 = \Theta(n^3)$. *Why:* at least half the bulk of $f + g$ comes from the larger, so dropping the smaller loses at most a factor of $1/2$ — a constant.

Multiplication — constants vanish, functions multiply.

$$O(c \cdot f(n)) = O(f(n)) \qquad \text{for constant } c > 0$$
$$O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n))$$

(Same for Ω, Θ.) c must be **strictly** positive — multiplying by zero wipes out any function.

> *When two functions in a product are increasing, **both** are important. An $O(n! \log n)$ function dominates $n!$ by just as much as $\log n$ dominates 1.*

The practical translation

CODE SHAPE	COMPLEXITY
two sequential blocks costing f and g	$\Theta(\max(f, g))$
loop of n iterations, body costs f	$\Theta(n \cdot f)$
nested loops	**multiply**
`if/else` with branches costing f, g	$O(\max(f, g))$ worst case
recursion	write a **recurrence** (M03)

Transitivity proof (the template for all such proofs)

Given $f(n) \le c_1 g(n)$ for $n > n_1$ and $g(n) \le c_2 h(n)$ for $n > n_2$, cascade:

$$f(n) \le c_1 \cdot g(n) \le c_1 c_2 \cdot h(n) \quad \text{for } n > n_3 = \max(n_1, n_2)$$

so $f = O(h)$ with $c_3 = c_1 c_2$. ∎ [Skiena §2.4.2, p.40]

Identities worth knowing [CLRS Problem 3-5]

```
Θ(Θ(f))                = Θ(f)
Θ(f) + O(f)            = Θ(f)
Θ(f) + Θ(g)            = Θ(f + g)
Θ(f) · Θ(g)            = Θ(f · g)
(a₁n)^{k₁} lg^{k₂}(a₂n) = Θ(n^{k₁} lg^{k₂} n)
f(n) + o(f(n))         = Θ(f(n))
```

Conjectures that are FALSE [CLRS Problem 3-4] — check yourself against these

CLAIM	VERDICT
`f = O(g) ⟹ g = O(f)`	**false** (`n = O(n²)` but `n² ≠ O(n)`)
`f + g = Θ(min(f, g))`	**false** — it's `Θ(max)`
`f = O(g) ⟹ 2^{f} = O(2^{g})`	**false** (`2n = O(n)` but `2^{2n} ≠ O(2ⁿ)`)
`f = Θ(f(n/2))`	**false** (`2ⁿ` vs `2^{n/2}`)
`f = O(f²)`	**false** in general — fails when `f < 1`, e.g. `f(n) = 1/n`
`f = O(g) ⟹ g = Ω(f)`	**true** (transpose symmetry)
`f = O(g) ⟹ lg f = O(lg g)` (with `lg g ≥ 1`, `f ≥ 1`)	**true**

---

7. Summations

Why they matter

Two reasons [Skiena §2.6, p.46]: they arise directly from loop analysis, and proving their closed forms is the classic application of induction. CLRS puts the same point structurally: *"When an algorithm contains an iterative control construct, you can express its running time as the sum of the times spent on each execution of the body."*

The essential closed forms

SERIES	CLOSED FORM	ASYMPTOTIC	SOURCE
Constant	$\Sigma_{i=1}^{n} 1 = n$	$\Theta(n)$	—
Arithmetic	$\Sigma_{k=1}^{n} k = n(n+1)/2$	$\Theta(n^2)$	CLRS A.1–A.2
General arithmetic	$\Sigma_{k=1}^{n} (a + bk)$	$\Theta(n^2)$ $(b > 0)$	CLRS A.3
Squares	$\Sigma_{k=0}^{n} k^2 = n(n+1)(2n+1)/6$	$\Theta(n^3)$	CLRS A.4
Cubes	$\Sigma_{k=0}^{n} k^3 = n^2(n+1)^2/4$	$\Theta(n^4)$	CLRS A.5
Powers	$S(n,p) = \Sigma_{i=1}^{n} i^p$	$\Theta(n^{p+1})$ for $p \geq 0$	Skiena §2.6
Geometric	$\Sigma_{k=0}^{n} x^k = (x^{n+1} - 1)/(x - 1)$, $x \neq 1$	$\Theta(x^{n+1})$ if $x > 1$	CLRS A.6
Infinite geometric	$\Sigma_{k=0}^{\infty} x^k = 1/(1 - x)$, $\lvert x \rvert < 1$	$\Theta(1)$	CLRS A.7
Harmonic	$H_n = \Sigma_{k=1}^{n} 1/k = \ln n + O(1)$	$\Theta(\log n)$	CLRS A.8–A.9
Harmonic, tight	$\ln(n+1) \leq H_n \leq \ln n + 1$		CLRS A.10
Differentiated geometric	$\Sigma_{k=0}^{\infty} k\,x^k = x/(1-x)^2$, $\lvert x \rvert < 1$	$\Theta(1)$	CLRS A.11
Telescoping	$\Sigma_{k=1}^{n} (a_k - a_{k-1}) = a_n - a_0$		CLRS A.12
Product→sum	$\lg \prod a_k = \Sigma \lg a_k$		CLRS

The three insights, not the formulas

1. Powers of integers: the sum is one degree higher. Sum of the first n integers is quadratic; sum of squares is cubic; sum of cubes is quartic. From the big-picture perspective, "the important thing is that the sum is quadratic, not that the constant is 1/2."

2. Geometric series are the great free lunch.

> *When $\lvert a \rvert < 1$, $G(n,a)$ converges to a constant as $n \to \infty$. This series convergence proves to be the great "free lunch" of algorithm analysis.* ***It means that the sum of a linear number of things can be constant, not linear.*** *For example, $1 + 1/2 + 1/4 + 1/8 + \ldots \leq 2$ no matter how many terms we add up. [Skiena §2.6, p.47]*

And in the other direction, when `a > 1`, the sum is dominated by its **last** term: `1+2+4+8+16+32 = 63 ≈ 2·32`. So `G(n,a) = θ(a^{n+1})`.

This single fact explains the `θ(n)` cost of building a heap (M05), the amortized `O(1)` of dynamic array growth (M09), and why a recursion tree with geometrically shrinking level costs is dominated by its root (M03).

3. Harmonic numbers are "where the log comes from."

> *The Harmonic numbers prove important, because they usually explain "where the log comes from" when one magically pops out from algebraic manipulation. For example, the key to analyzing the average-case complexity of quicksort is the summation* $\sum_{i=1}^{n} 1/i$*. Employing the Harmonic number's* θ *bound immediately reduces this to* $\theta(n \log n)$*. [Skiena §2.7.6, p.51]*

Techniques for manipulating sums

Telescoping. Rewrite terms so consecutive pieces cancel:

```
Σ_{k=1}^{n-1} 1/(k(k+1)) = Σ (1/k − 1/(k+1)) = 1 − 1/n
```

→ **C++ implementation:** A5 The summation closed forms

Reindexing. *"If the summation index appears in the body of the sum with a minus sign, it's worth thinking about reindexing."* [CLRS §A.1, p.1143]

```
Σ_{k=1}^{n} 1/(n − k + 1)   ── set j = n − k + 1 ──►   Σ_{j=1}^{n} 1/j = H_n
```

Separating the largest term — the workhorse of inductive proofs of summations. Skiena's example, proving `Σ_{i=1}^{n} i·i! = (n+1)! − 1`:

```
Σ_{i=1}^{n+1} i·i!  = (n+1)·(n+1)! + Σ_{i=1}^{n} i·i!      ← reveals the hypothesis
                    = (n+1)·(n+1)! + (n+1)! − 1
                    = (n+1)!·((n+1) + 1) − 1
                    = (n+2)! − 1                            ∎
```

> *This general trick of **separating out the largest term** from the summation to reveal an instance of the inductive assumption lies at the heart of all such proofs. [Skiena §2.6, p.47]*

Linearity, including with asymptotic notation:

```
Σ_{k=1}^{n} θ(f(k)) = θ( Σ_{k=1}^{n} f(k) )
```

Note the subtlety CLRS flags: on the left `θ` applies to `k`; on the right it applies to `n`.

Nested loops → nested summations

Matrix multiplication [Skiena §2.5.4, p.45]:

```
// A is x-by-y, B is y-by-z, C is x-by-z.
// `const vector<vector<double>>&` = call-by-constant-reference: no
copy of a
// potentially huge matrix, and the const forbids us from modifying
the inputs.
// C is a plain `&` (call-by-reference) precisely because we DO
write to it.
void matrixMultiply(const vector<vector<double>>& A,
                    const vector<vector<double>>& B,
                    vector<vector<double>>& C) {
    const int x = (int)A.size();                 // rows of A
    const int y = (int)B.size();                 // rows of B ==
columns of A
    const int z = y ? (int)B[0].size() : 0;      // columns of B
    C.assign(x, vector<double>(z, 0.0));         // x rows, each z
zeros

    for (int i = 0; i < x; ++i)
        for (int j = 0; j < z; ++j) {
            C[i][j] = 0;
            for (int k = 0; k < y; ++k)
                C[i][j] += A[i][k] * B[k][j];
        }
}
```

```
M(x,y,z) = Σ_{i=1}^{x} Σ_{j=1}^{y} Σ_{k=1}^{z} 1
```

→ **C++ implementation:** A4 Nested loops become nested summations

Evaluate from the right inward: sum of `z` ones is `z`; sum of `y` `z`'s is `yz`; sum of `x` `yz`'s is `xyz`. So `Θ(xyz)`, and `Θ(n³)` when all dimensions are `n`. Both `O` and `Ω` hold because the loop bounds are fixed by the dimensions — no early exit. Faster algorithms exist (Strassen, M03).

---

8. Logarithms

The definition and the three bases

`b^x = y ⟺ x = log_b y ⟺ b^{log_b y} = y`.

BASE	NOTATION	WHERE IT COMES FROM
2	`lg n`	**binary logarithm** — repeated halving, tree nodes, bits. Most algorithmic logs.
`e ≈ 2.71828`	`ln n`	natural log; inverse of `exp`; appears in analysis via integrals and `H_n`
10	`log n`	common log; slide rules, historical

CLRS notation conventions [§3.3, p.66]: `lg^k n = (lg n)^k` (exponentiation), `lg lg n = lg(lg n)` (composition). And: *in the absence of parentheses, a logarithm applies only to the next term*, so `lg n + 1` means `(lg n) + 1`, not `lg(n+1)`.

Identities

```
a = b^{log_b a}
log_c(ab)   = log_c a + log_c b
log_b(a^n)  = n log_b a
log_b a     = log_c a / log_c b          ← change of base
log_b(1/a)  = −log_b a
log_b a     = 1 / log_a b
a^{log_b c} = c^{log_b a}                ← surprisingly useful
```

Plus the exponential facts: `1 + x ≤ e^x` (equality only at `x = 0`), `e^x = 1 + x + Θ(x²)` as `x → 0`, and `lim_{n→∞}(1 + x/n)ⁿ = e^x`.

Two consequences that matter algorithmically [Skiena §2.8, p.53]

1. The base doesn't matter inside `O`. Changing base multiplies by `log_c a`, a constant, absorbed by the notation.

```
log₂(10⁶) = 19.93     log₃(10⁶) = 12.58     log₁₀₀(10⁶) = 3
```

A huge change in base makes little difference in value.

> ***Stop and Think:*** *how many queries does binary search take on the million-name Manhattan phone book if each split were 1/3-to-2/3 instead of 1/2-to-1/2?* `log_{3/2}(10⁶) ≈ 35` *instead of* `log₂(10⁶) ≈ 20`*. Not a significant change.* ***The effectiveness of binary search comes from its logarithmic running time, not the base of the log.***

2. Logarithms cut any function down to size. `log_a n^b = b·log_a n`, so the log of any polynomial is `O(lg n)`. *"Performing a binary search on a sorted array of* `n²` *things requires only twice as many comparisons as a binary search on* `n` *things."*

And it is how you do arithmetic on factorials at all:

```
n! = ∏_{i=1}^{n} i   →   lg n! = Σ_{i=1}^{n} lg i = θ(n log n)
```

Where logarithms come from — the five sources [Skiena §2.7]

SOURCE	MECHANISM
Binary search	The number of times you can halve `n` until 1 is `log₂ n`. 20 comparisons find any name in a million-name phone book.
Tree height	A tree of degree `d` and height `h` has up to `d^h` leaves. `n = d^h ⟹ h = log_d n`. "**Short trees can have very many leaves**, which is the main reason binary trees prove fundamental to the design of fast data structures."
Bit width	Bit patterns double with each added bit, so representing `n` possibilities needs `w` bits where `2^w = n`, i.e. `w = log₂ n`.
Multiplication / exponentiation	`log(xy) = log x + log y`; `a^b = exp(b·ln a)`.
Harmonic sums	`Σ 1/i = θ(log n)`.

> ***Take-Home Lesson:*** *Logarithms arise whenever things are* ***repeatedly halved or doubled****.*

Algorithm: Fast Exponentiation (binary exponentiation)

Problem. Compute a^n exactly for large n (primality testing, cryptography, modular arithmetic). Precision issues rule out `exp(n ln a)`.

Core intuition. $n = \lfloor n/2 \rfloor + \lceil n/2 \rceil$. If n is even, $a^n = (a^{n/2})^2$; if odd, $a^n = a \cdot (a^{\lfloor n/2 \rfloor})^2$. Each step halves the exponent for at most two multiplications.

Mental model. *Square the base, halve the exponent — the binary representation of* n *tells you where to multiply in the extra factor.*

Pseudocode [Skiena §2.7.5, p.51]:

```
power(a, n)
    if n = 0: return 1
    x = power(a, ⌊n/2⌋)
    if n is even: return x²
    else:         return a · x²
```

→ **C++ implementation:** A1 power

Correctness. Strong induction on n. Base $n = 0$ gives 1. For $n > 0$, by hypothesis $x = a^{\lfloor n/2 \rfloor}$. If n even, $n = 2\lfloor n/2 \rfloor$ so $x^2 = a^n$. If n odd, $n = 2\lfloor n/2 \rfloor + 1$ so $a \cdot x^2 = a^n$. ∎

Complexity. $T(n) = T(n/2) + O(1)$ → $\mathbf{O(\log n)}$ **multiplications**, versus $n - 1$ naively.

> *This simple algorithm illustrates an important principle of divide and conquer.* ***It always pays to divide a job as evenly as possible.*** *When* n *is not a power of two, a difference of one element between the two sides cannot cause any serious imbalance.*

C++ Implementation (iterative, the competitive-programming standard):

```cpp
#include <cstdint>

// Computes (base^exp) mod m in O(log exp) multiplications.
// Requires m > 0. Uses __int128 to avoid overflow on the multiply.
```

```
uint64_t powMod(uint64_t base, uint64_t exp, uint64_t m) {
    uint64_t result = 1 % m;
    base %= m;
    while (exp > 0) {
        if (exp & 1ULL)                                    // bit
set -> fold base in
            result = static_cast<uint64_t>(
                        (static_cast<__uint128_t>(result) * base)
% m);
        base = static_cast<uint64_t>(
                    (static_cast<__uint128_t>(base) * base) % m);
        exp >>= 1;
    }
    return result;
}
```

Implementation notes.

- `result = 1 % m` handles `m = 1` correctly (answer 0), a classic off-by-one.
- `__int128` for the intermediate product: `u64 × u64` overflows. Without a 128-bit type, use `unsigned long long` with a modulus under 2^{31}, or Montgomery multiplication.
- The iterative form avoids `O(log n)` stack frames and is the one to write in an interview.
- The same skeleton generalizes to **matrix exponentiation** (linear recurrences in $O(k^3 \log n)$) and to any associative operation — this is the "binary lifting" pattern that reappears in LCA (M08) and sparse tables.

Common bugs. Forgetting `% m` on the initial `result`; squaring *after* using `base` in an iteration where the bit was set (order matters only in that you must square every iteration); overflow.

Recognition pattern. Any time you must apply an **associative** operation `n` times and `n` is large: modular powers, Fibonacci via matrix power, `n`-th ancestor in a tree, transitive closure powers.

---

9. War Story: Mystery of the Pyramids — algorithmic vs hardware speedup

[Skiena §2.9, p.54]

The problem. For every `k` from 1 to 10^9, find the minimum number of *pyramidal numbers* `(m³ − m)/6` summing to `k` (conjectured ⩽ 5 since 1928).

The client's algorithm. There are `θ(n^{1/3})` pyramidal numbers below `n` (1816 of them below 10^9). For each `k`, brute-force test sums of two, then three, then four, then five. Since ~45% of integers need three and most of the rest need four, the typical run does all the three-tests: `O(n · (n^{1/3})³) = O(n²)`. It died at `n ≈ 100,000`.

The fix — model it as knapsack, precompute pairs. Build a sorted **two-table** of all sums of two pyramidal numbers (at most `1816² ≈ 3.3M`, about half that after dedup). Then:

- is `k` pyramidal? → check the 1816-element list
- sum of two? → binary search the two-table
- sum of three? → for each of 1816 values `p[i]`, binary search for `k − p[i]` → `O(n^{1/3} lg n)`
- sum of four? → search for `k − two[i]`; terminates fast since almost every `k` has many four-representations

Total: **`O(n^{4/3} lg n)`** vs `O(n²)` — about **30,000× faster** at `n = 10⁹`.

The moral, quantified:

> *His million-dollar computer had 16 processors, each reportedly five times faster on integer computations than the $3,000 machine on my desk. That gave him a maximum potential speedup of **less than 100 times**. Clearly, the algorithmic improvement was the big winner here, as it is certain to be in any sufficiently large computation.*

Two engineering asides Skiena adds, both worth internalizing: turning on the compiler optimizer took the rerun from 1.113s to 0.334s (*"this is why you need to remember to turn your optimizer on"*), and 25 years of free hardware improvement gave hundreds of times more speed for zero work — but still nowhere near the 30,000× from the algorithm.

---

10. On the RAM model's honesty

Skiena's defense of the model is worth keeping because it is the right attitude toward all modeling:

> *Multiplying two numbers takes more time than adding on most processors, which violates the first assumption. Fancy compiler loop unrolling and hyperthreading may well violate the second. And memory-access times differ greatly depending on where your data sits in the storage hierarchy. This makes us **zero for three** on the truth of our basic assumptions. And yet, despite these objections, the RAM proves an excellent model. … Every model in science has a size range over which it is useful. Take the model that the Earth is flat. … when laying the foundation of a house, the flat Earth model is sufficiently accurate that it can be reliably used. [Skiena §2.1, p.32]*

> ***It is difficult to design an algorithm where the RAM model gives you substantially misleading results.***

Chapter in One Page

CONCEPT	THE ONE-LINE VERSION
Why asymptotics	Exact functions have too many bumps and too much irrelevant detail.
O	upper bound, $\exists c$: $f \le cg$. Like $\le$.
Ω	lower bound, $\exists c$: $f \ge cg$. Like $\ge$.
Θ	tight, $\exists c_1, c_2$: $c_1 g \le f \le c_2 g$. Like $=$.
o / ω	strict; the bound holds **for all** c, not some. Like $<$ / $>$.
Theorem 3.1	$\Theta \iff O$ **and** Ω.
The precision rule	Never drop "worst-case" from a Θ claim.
The conflation trap	$O(n \log n)$ is *not* necessarily faster than $O(n^2)$ — O is only an upper bound.
Proving Θ	The O is loop counting; the Ω requires **constructing a bad instance**.
Round-it-up heuristic	Multiply max iterations of nested loops. Always correct, sometimes pessimistic.
Addition rule	$f + g = \Theta(\max(f, g))$.
Multiplication rule	Constants vanish; increasing functions multiply.
Dominance	$n! \gg c^n \gg n^3 \gg n^2 \gg n^{1+\varepsilon} \gg n \log n \gg n \gg \sqrt{n} \gg \log^2 n \gg \log n \gg \log \log n \gg \alpha(n) \gg 1$
Combinatorial → complexity	pairs→n^2, triples→n^3, subsets→2^n, orderings→$n!$

Trichotomy	**fails** — `n` vs `n^{1+sin n}` are incomparable.
Arithmetic series	`n(n+1)/2 = Θ(n²)`; `Σ i^p = Θ(n^{p+1})`.
Geometric, `x < 1`	converges to a **constant** — "the great free lunch".
Geometric, `x > 1`	`Θ(x^{n+1})` — dominated by the last term.
Harmonic	`H_n = Θ(log n)` — "where the log comes from".
Telescoping	`Σ(a_k − a_{k−1}) = a_n − a₀`.
Nested loops	nested summations, **evaluate from the inside out**.
Log base	irrelevant inside `O` — change of base is a constant factor.
Logs come from	halving · tree height · bit width · `lg n!` · harmonic sums.
`lg n!`	`Θ(n log n)` via Stirling. Basis of the sorting lower bound.
Fast exponentiation	`Θ(log n)` multiplications; the binary-lifting pattern.
`lg* n`	`≤ 5` for every `n` in this universe.
Algorithms beat hardware	30,000× from an algorithm vs < 100× from a supercomputer.

Recognition Table

CLUE	WHAT TO DO
Two independent nested loops over `n`	multiply → `n²`
Loop where the index doubles / halves	`log n` factor
Loop over all pairs / all triples	`n²` / `n³`
Enumerating subsets	`2ⁿ` — check `n ≤ 20`
Enumerating permutations	`n!` — check `n ≤ 11`
Sum of `1/i` shows up	harmonic → `Θ(log n)`
Sum where each term is half the last	geometric → `Θ(1)`, work is dominated by the **first** term
Sum where each term is double the last	geometric → dominated by the **last** term
Recursive work shrinking by a constant factor per level	recursion tree, see M03
Need a `Θ`, have an `O`	construct the adversarial instance
Constraint says `n ≤ 10⁵`	intended solution is `O(n log n)` or `O(n √n)`
Constraint says `n ≤ 5000`	`O(n²)` is intended
Constraint says `n ≤ 20`	bitmask / `2ⁿ` DP intended
`α(n)` in a bound	union-find is involved
`log n / log log n`	tree of degree `log n`

Common Mistakes Recap

1. Saying "algorithm A is `Θ(n²)`" when only its **worst case** is.
2. Using `O` where you mean `Θ`, then comparing two algorithms by their `O` bounds.
3. Writing "at least `O(n²)`" — meaningless.
4. Rejecting a valid bound because the required constant is huge or tiny.
5. Assuming `f = O(g) ⟹ 2^f = O(2^g)`.
6. Assuming `f + g = Θ(min(f,g))`.
7. Assuming any two functions are comparable (trichotomy fails).
8. Forgetting the `Ω` half when asked for a tight bound.
9. Treating the log base as significant.
10. Multiplying loop bounds when the inner loop's *total* work is globally bounded

(over-estimates; see amortized analysis, M09).

11. Forgetting that a geometric series with ratio < 1 sums to a **constant**, not to `n` terms of work.

Self-Test

1. Give the formal set definitions of all five notations. What is the single quantifier difference between `O` and `o`? *(§2)*
2. Prove `4n² + 100n + 500 = O(n²)` by exhibiting `c` and `n₀`. Then prove `n³ − 100n² ∉ O(n²)`. *(§2)*
3. Why is "insertion sort's running time is `Θ(n²)`" wrong, while "insertion sort's running time is `O(n²)`" is right? *(§3)*
4. State what `Ω(n²)` on a worst-case running time actually asserts — carefully. *(§4)*
5. Give CLRS's thirds argument for insertion sort's `Ω(n²)` lower bound. *(§4)*
6. Why is selection sort `Θ(n²)` with no case analysis at all? *(§4)*
7. Simplify `O(n + m + (n−m)(m+2))` to `O(nm)`, justifying each step. Then give the instance forcing `Ω(nm)`. *(§4)*
8. Rank: `n^{1/lg n}`, `lg*n`, `(lg n)!`, `n lg n`, `2^{√(2 lg n)}`, `n!`, `4^{lg n}`, `lg(n!)`. *(§5)*
9. Which of these are true? `f = O(g) ⟹ g = O(f)` · `f + g = Θ(min)` · `f = O(g) ⟹ 2^f = O(2^g)` · `f = Θ(f(n/2))` *(§6)*
10. Give closed forms for arithmetic, geometric (both cases), and harmonic series, with their `Θ`s. *(§7)*
11. Why is a convergent geometric series "the great free lunch of algorithm analysis"? Name two algorithms whose analysis depends on it. *(§7)*
12. Evaluate `Σ_{i=1}^{x} Σ_{j=1}^{y} Σ_{k=1}^{z} 1` from the inside out. *(§7)*
13. Prove `Σ_{i=1}^{n} i·i! = (n+1)! − 1` by induction. What is the general trick? *(§7)*
14. Name the five places logarithms come from. *(§8)*
15. Why doesn't the base of a logarithm matter? How many binary-search probes on `10⁶` items with a 1/3–2/3 split? *(§8)*
16. Write iterative `powMod` from memory. Why `1 % m` and not `1`? *(§8)*
17. Show `lg(n!) = Θ(n log n)`, and say which lower bound depends on it. *(§5, §8)*

Practice — where to drill this module

M02 is an *analysis* module, so most of the drilling is "state the complexity and defend it" rather than "write the code". These are the problems where the analysis is the hard part.

IDEA IN THIS MODULE	PROBLEM	WHY IT'S THE RIGHT DRILL
Fast exponentiation	50 · Pow(x, n)	the exact `power(a,n)` of §8 — and `n = INT_MIN` is the edge case the pseudocode hides
Modular fast exponentiation	372 · Super Pow	forces the modular version, plus a digit-by-digit outer loop
Naive matching and its `Θ(nm)`	28 · Find the Index of the First Occurrence in a String	write the naive version, then explain why it is `Θ(nm)` and construct the bad input
Where a log comes from (halving)	704 · Binary Search · 33 · Search in Rotated Sorted Array · 153 · Find Minimum in Rotated Sorted Array	three problems whose whole content is "the answer is `lg n` because you halve"
Rolling-hash vs naive cost	187 · Repeated DNA Sequences	the naive answer is `Θ(nm)`; the intended one is `Θ(n)`. The gap *is* this module
Nested loops → nested sums	*(no single problem — write your own)*	take any `O(n³)` brute force you already have and count its innermost executions exactly; the `A4` entry below shows how

Beyond LeetCode. CSES Problem Set — *Introductory Problems* and *Mathematics*. Codeforces `math` tag and `binary search` tag.

The drill that actually matters here is not a problem at all: take any solution you have already written and (a) write down its exact operation count as a summation, (b) evaluate the summation in closed form, (c) construct an input that forces the worst case. The appendix below does exactly that for four algorithms, with the counts measured rather than asserted.

C++ Toolkit for This Module

Language material from Weiss, ***Data Structures and Algorithm Analysis in C++****, 4th ed., ch. 1.*

1. Integer types, width, and overflow — the analysis module's own bug class

Asymptotics tells you n^2 grows fast; C++ then quietly wraps around when it does.

TYPE	WIDTH	MAX	OVERFLOW BEHAVIOUR
`int`	32 bits	$2\,147\,483\,647$	**undefined behaviour** — the optimizer may assume it never happens
`long long`	64 bits	$\approx 9.22 \times 10^{18}$	undefined behaviour
`unsigned` / `size_t`	32 / 64	—	**defined**: wraps modulo 2^n
`__int128`	128 (GCC/Clang ext.)	$\approx 1.7 \times 10^{38}$	for the intermediate product in modular arithmetic

Signed overflow being *undefined* rather than *wrapping* is the part people get wrong. `if (a + b < a)` is not a valid overflow check for signed types — the compiler is entitled to delete it. Use a wider type, or `__builtin_add_overflow`.

The rule of thumb this module gives you: **if your loop count is $O(n^2)$ and n can reach 10^5, the count is 10^{10} and does not fit in `int`.** Every counter in the appendix below is `long long` for that reason.

2. `size_t` and the unsigned trap

`vector::size()` returns `size_t`, an **unsigned** type. Mixing it with `int` triggers the usual-arithmetic-conversions rule: the `int` is converted to unsigned.

```
void unsignedTrap(const vector<int>& v) {
    // BROKEN when v is empty: v.size() - 1 is 0u - 1 == SIZE_MAX
    for (size_t i = 0; i + 1 < v.size(); ++i) { /* correct rewrite
*/ }
    // and this comparison warns under -Wall -Wextra for a good
reason:
    const int n = (int)v.size();      // cast ONCE, at the top,
then use int
    for (int i = 0; i < n; ++i) { (void)v[i]; }
}
```

Weiss's convention throughout is to take the size into an `int` once. Do the same, and `-Wsign-compare` stays quiet.

3. `double` is not a real number

The summation checks in `A5` compare a loop's result against a closed form. They cannot use `==`:

```
// Relative tolerance, not absolute: 1e-9 is a huge error next to
1e20
// and an impossible one next to 1e-20.
bool nearly(double a, double b, double tol) {
    return fabs(a - b) <= tol * max(1.0, fabs(b));
}
```

`double` has 53 bits of mantissa ≈ 15–16 decimal digits. Summing `1/k` for `k` up to 10^6 accumulates rounding, which is why the harmonic check below uses a `1e-5` tolerance while the arithmetic one uses `1e-12`.

4. `std::function` — type erasure, and what it costs

`A6` stores growth-rate functions in a table, so it needs a type that can hold *any* callable:

```
double crossoverDemo(const function<double(double)>& f, double n) {
return f(n); }
```

`function<double(double)>` accepts a lambda, a function pointer, or a functor — but it does so through a **virtual call and possibly a heap allocation**. A template parameter (`template <typename F>`) is the zero-overhead alternative when the type is known at compile time. Use `std::function` when you must store heterogeneous callables in one container, as here; use a template when you are just passing a comparator to `sort`.

5. Passing an out-parameter: pointer vs reference

`A3` needs to return two things — a comparison count and a match position. Three idioms, in order of preference:

```
struct MatchResult { long long compares; int where; };   // 1.
best: a named struct
MatchResult matchA(const string& p, const string& t);
void matchB(const string& p, const string& t, int& where);   // 2.
reference out-param
void matchC(const string& p, const string& t, int* where);   // 3.
pointer out-param
```

Weiss's guidance [§1.5.3] favours references over pointers, because a reference cannot be null and needs no `*` at the call site. The one argument for the pointer form is exactly that: `matchC(p, t, &where)` makes it **visible at the call site** that `where` will be modified. The appendix uses the pointer form for that reason, and says so.

6. Structured bindings (C++17)

```
void bindingsDemo() {
    vector<array<int,3>> dims = {{4,5,6},{10,10,10}};
    for (auto [x, y, z] : dims) (void)(x + y + z);   // unpacks
each array into three names
}
```

This works on arrays, `pair`, `tuple`, and any struct with public members. It replaces `d[0], d[1], d[2]` with names that say what they mean — and in this module that matters, because `x`, `y`, `z` are the three loop bounds whose product is the running time.

Appendix — C++ for Every Pseudocode Block

M02 has one procedure in pseudocode (`power`) and several *analysis claims* stated as summations. Each entry below turns one of those into runnable C++ that **measures** the quantity the text asserts, so nothing here has to be taken on faith.

A1 power

Pseudocode: §8, "Algorithm: Fast Exponentiation (binary exponentiation)".

```
// LITERAL translation of Skiena's pseudocode: recursive, one
recursive call.
//
//   power(a, n)
//       if n = 0: return 1
//       x = power(a, floor(n/2))
//       if n is even: return x^2
//       else:         return a * x^2
//
// `long long` throughout: a^n overflows 32-bit almost immediately,
and signed
// overflow is UNDEFINED behaviour, not wraparound.
long long powerRecursive(long long a, long long n) {
    if (n == 0) return 1;                    // if n = 0: return 1
    long long x = powerRecursive(a, n / 2);   // x = power(a,
floor(n/2))
    // CRITICAL: `x` is computed ONCE and squared. Writing
    //     return powerRecursive(a,n/2) * powerRecursive(a,n/2);
    // is the same mathematics and a catastrophically different
algorithm --
    // it turns T(n) = T(n/2) + O(1)  =  Theta(lg n)
    //     into T(n) = 2T(n/2) + O(1) =  Theta(n).
    // This is the single most instructive bug in the module.
    if (n % 2 == 0) return x * x;            // n even
    return a * x * x;                        // n odd
}

// Counts the multiplications powerRecursive performs, without
doing them.
// Even step: one multiply (x*x). Odd step: two (x*x, then a*
(x*x)).
int powerMultiplyCount(long long n) {
    if (n == 0) return 0;
    return powerMultiplyCount(n / 2) + (n % 2 == 0 ? 1 : 2);
```

```
}
```

Complexity. `T(n) = T(n/2) + O(1) = Θ(lg n)` multiplications; `Θ(lg n)` recursion depth. The iterative `powMod` in the body of §8 is the same algorithm with the recursion unrolled into a scan over the bits of `n` — and it is `Θ(1)` space.

> *Verified: `powerRecursive` agrees with a naive repeated-multiplication loop for all `a ∈ [0,5]`, `n ∈ [0,12]`. Multiplication counts, against the naive `n − 1`:*
>
> | N | LG N | MULTIPLICATIONS | NAIVE |
> |---|---|---|---|
> | 10 | 3 | 6 | 9 |
> | 1 000 | 10 | 16 | 999 |
> | 10^6 | 20 | 27 | 999 999 |
> | 10^9 | 30 | **43** | 999 999 999 |
>
> *Forty-three multiplications instead of a billion. That is what `Θ(lg n)` buys.*

A2 Selection sort's exact comparison count

Corresponds to the summation `T(n) = Σ_{i=0}^{n-1} Σ_{j=i+1}^{n-1} 1 = n(n-1)/2` in §4.

```
// The same selectionSort as in the body, with a counter threaded
through, so
// the summation in the text can be CHECKED rather than believed.
//
// Return type is `long long`, not `int`: for n = 100000 the count
is
// n(n-1)/2 ~ 5e9, which overflows a 32-bit int. This is the
module's own
// lesson applied to the module's own code.
long long selectionSortCompares(vector<int>& s) {
    const int n = (int)s.size();       // one cast, at the top (see
toolkit §2)
    long long compares = 0;
    for (int i = 0; i < n; ++i) {                 // outer: i = 0
.. n-1
        int mn = i;
```

```
        for (int j = i + 1; j < n; ++j) {          // inner: j = i+1
.. n-1, i.e. n-i-1 times
            ++compares;
            if (s[j] < s[mn]) mn = j;
        }
        // std::swap: for `int` this is three moves; for a big type
it is three
        // MOVES rather than three copies in C++11 [Weiss 1.5.5,
p.29].
        // swap(s[i], s[i]) when mn == i is harmless but not free -
- guarding it
        // with `if (mn != i)` is a real micro-optimisation for
expensive types.
        swap(s[i], s[mn]);
    }
    return compares;
}
```

The point. `Σ_{i=0}^{n-1} (n − i − 1) = (n−1) + (n−2) + … + 1 + 0 = n(n−1)/2`. Selection sort is the rare algorithm with **no case analysis at all**: the loop bounds do not depend on the data, so best = average = worst = `θ(n²)` exactly.

> *Verified: for `n ∈ {0, 1, 2, 10, 100, 500}`, on random, already-sorted, and reverse-sorted inputs, the measured count was **exactly** `n(n−1)/2` **every time** — and the output always matched `std::sort`.*

A3 Naive string matching, and its `Ω(nm)` instance

Corresponds to the simplification chain and the `(n − m + 1)·m = Ω(nm)` block in §4.

```
// Same findMatch as the body, instrumented. Reports BOTH the
comparison count
// and the match position, so the out-parameter question of toolkit
5 is live.
//
// The pointer form `int* whereFound` is chosen deliberately: it
forces the
// caller to write `&where`, which makes the mutation visible at
the call site.
```

```
long long findMatchCompares(const string& p, const string& t, int*
whereFound) {
    const int m = (int)p.size();
    const int n = (int)t.size();
    long long compares = 0;
    *whereFound = -1;                          // "not found" -- set
BEFORE any early return

    // `i + m <= n` not `i < n - m`: with n and m as ints this is
equivalent,
    // but if they were size_t, `n - m` on n < m would wrap to a
huge number and
    // the loop would run off the end of the string. Write the
addition form.
    for (int i = 0; i + m <= n; ++i) {
        int j = 0;
        while (j < m) {
            ++compares;                        // count the character
comparison itself
            if (t[i + j] != p[j]) break;
            ++j;
        }
        if (j == m) { *whereFound = i; return compares; }
    }
    return compares;
}
```

Building the worst case. `t = "aaa…a"` (`n` a's) and `p = "aa…ab"` (`m−1` a's then `b`). Every one of the `n − m + 1` alignments matches `m − 1` characters and fails on the last, so the count is exactly `(n − m + 1)·m`.

> *Verified, with `m = 5`:*
>
> | N | MEASURED COMPARISONS | (N − M + 1)·M |
> |---|---|---|
> | 50 | 230 | 230 |
> | 100 | 480 | 480 |
> | 200 | 980 | 980 |
> | 400 | 1980 | 1980 |

> *Exact agreement, so the `Ω(nm)` lower bound is not an estimate. On an ordinary input (`p = "abc"` in `t = "xxabcxx"`) it finds the match at index 2 after **5** comparisons — which is why naive matching is fine in practice and terrible in theory.*

A4 Nested loops become nested summations

Corresponds to `M(x,y,z) = Σ_{i=1}^{x} Σ_{j=1}^{y} Σ_{k=1}^{z} 1` in §4.

```
// Counts executions of the innermost statement of the triple loop,
to check
// that the triple summation really evaluates to x*y*z.
long long matrixMultiplyOps(int x, int y, int z) {
    long long ops = 0;
    // vector<vector<double>> is a vector OF vectors: rows are
separate
    // allocations, so it is not contiguous and each A[i][k] is two
    // dereferences. A single vector<double> of size x*y with
manual indexing
    // (A[i*y + k]) is measurably faster -- but this shape matches
the
    // pseudocode, and clarity wins in notes.
    vector<vector<double>> A(x, vector<double>(y, 1.0));
    vector<vector<double>> B(y, vector<double>(z, 1.0));
    vector<vector<double>> C(x, vector<double>(z, 0.0));

    for (int i = 0; i < x; ++i)
        for (int j = 0; j < z; ++j) {
            C[i][j] = 0;
            for (int k = 0; k < y; ++k) { ++ops; C[i][j] += A[i][k]
* B[k][j]; }
        }
    return ops;
}
```

Evaluate from the inside out: the innermost sum of `z` ones is `z`; the middle sum of `y` copies of `z` is `yz`; the outer sum of `x` copies of `yz` is `xyz`. **`Θ(xyz)`**, and `Θ(n³)` when all three dimensions are `n`. Both `O` and `Ω` hold, because the loop bounds are fixed by the dimensions — there is no early exit.

> *Verified: `(x,y,z) = (4,5,6) → 120`; `(10,10,10) → 1000`; `(20,3,7) → 420`. Exactly `xyz` in each case.*

A5 The summation closed forms

Corresponds to the arithmetic / geometric / harmonic / telescoping identities in §6.

```
// Each function returns BOTH the value computed by brute-force
summation and
// the value from the closed form, so a test can compare them.
// A tiny struct beats pair<double,double>: `.direct` and `.closed`
say what
// they mean, `.first` and `.second` do not.
struct SumCheck { double direct, closed; };

// Arithmetic: sum_{k=1}^{n} k = n(n+1)/2  -->  Theta(n^2)
SumCheck arithmeticSum(int n) {
    double s = 0;
    for (int k = 1; k <= n; ++k) s += k;
    // `n * (n + 1.0) / 2.0`: the 1.0 forces DOUBLE arithmetic.
Written as
    // `n * (n + 1) / 2` with int n = 100000, the product 1e10
overflows int.
    return {s, n * (n + 1.0) / 2.0};
}

// Geometric: sum_{k=0}^{n} x^k = (x^{n+1} - 1)/(x - 1)
// For x > 1 the sum is Theta(largest term); for x < 1 it converges
to a
// CONSTANT, which is why such a sum is "free" in an analysis.
SumCheck geometricSum(double x, int n) {
    double s = 0;
    for (int k = 0; k <= n; ++k) s += pow(x, k);
    return {s, (pow(x, n + 1) - 1) / (x - 1)};
}

// Harmonic: H_n = sum_{k=1}^{n} 1/k = ln n + gamma + O(1/n) =
Theta(lg n)
SumCheck harmonicSum(int n) {
    double s = 0;
    for (int k = 1; k <= n; ++k) s += 1.0 / k;
    const double gamma = 0.5772156649015329;   // Euler-Mascheroni
    return {s, log((double)n) + gamma};
}
```

```
// Telescoping: sum_{k=1}^{n-1} 1/(k(k+1)) = sum (1/k - 1/(k+1)) =
1 - 1/n
SumCheck telescopingSum(int n) {
    double s = 0;
    // `k * (k + 1.0)`: again the 1.0. With int k near 46341, k*
(k+1) overflows.
    for (int k = 1; k <= n - 1; ++k) s += 1.0 / (k * (k + 1.0));
    return {s, 1.0 - 1.0 / n};
}

// sum_{i=1}^{n} i*i! = (n+1)! - 1   -- the CLRS Appendix A worked
example
SumCheck factorialWeightedSum(int n) {
    double s = 0, fact = 1;
    for (int i = 1; i <= n; ++i) { fact *= i; s += i * fact; }
    double f = 1;
    for (int i = 1; i <= n + 1; ++i) f *= i;
    return {s, f - 1};
}
```

> *Verified, direct summation against closed form:*
>
> | *SERIES* | *N* | *DIRECT* | *CLOSED FORM* |
> |---|---|---|---|
> | *arithmetic* | *1 000* | *500 500* | *500 500* |
> | *geometric* `x = 2` | *30* | *2 147 483 647* | *2 147 483 647* |
> | *geometric* `x = ½` | *60* | *2.000000* | *→ 2 (converges)* |
> | *harmonic* | 10^6 | *14.392727* | `ln n + γ` *= 14.392726* |
> | *telescoping* | 10^5 | *0.999990000* | `1 - 1/n` *= 0.999990000* |
> | `Σ i·i!` | *15* | *20 922 789 887 999* | `(n+1)! - 1` *= 20 922 789 887 999* |
>
> *Note the geometric row with* `x = 2`*: the sum is* `2 147 483 647` *while the* ***largest single term*** `2³⁰` *is* `1 073 741 824` *— a ratio of exactly 2. That is the whole content of "an increasing geometric series is* `Θ(its largest term)`*". And the* `x = ½` *row converges to 2 no matter how many terms you add: that is why a decreasing geometric series contributes only* `Θ(1)`*.*

A6 The dominance hierarchy, measured

Corresponds to `n! ≫ cⁿ ≫ n³ ≫ n² ≫ n^{1+ε} ≫ n log n ≫ n ≫ …` *in §5.*

```
// Given two growth functions expressed as their LOGARITHMS (so 2^n
does not
// overflow), binary-search for the n where g overtakes f.
//
// Precondition: f(lo) > g(lo) and f(hi) < g(hi) -- i.e. the
crossover is
// bracketed. The caller asserts this; a bisection with a bad
bracket silently
// returns nonsense, which is the classic bisection bug.
//
// std::function<double(double)> can hold any callable (toolkit 4).
Passed by
// const& to avoid copying the type-erased object on every call.
double crossover(const function<double(double)>& f, const
function<double(double)>& g,
                 double lo, double hi) {
    for (int it = 0; it < 200; ++it) {          // fixed iteration
count: no epsilon to tune,
        double mid = lo + (hi - lo) / 2;        // and doubles run
out of precision long before 200
        if (f(mid) < g(mid)) hi = mid;          // g has already
overtaken -> crossover is <= mid
        else                 lo = mid;
    }
    return lo;
}
// `lo + (hi - lo) / 2`, not `(lo + hi) / 2`: for doubles this is
about
// avoiding overflow to infinity when both are ~1e308; for ints it
is the
// famous binary-search overflow bug. Same habit, both worlds.
```

> *Verified crossovers (the smallest `n` at which the second function is larger):*
>
> | *GROWS SLOWER EVENTUALLY* | *OVERTAKEN BY* | *AT N =* |
> |---|---|---|
> | n^3 | 2^n | *10* |
> | $100n$ | n^2 | *101* |
> | n^{10} | 1.1^n | *686* |
> | $n \lg n$ | $n^{1.1}$ | $\approx 4.9 \times 10^{17}$ |

> *Read the first and last rows together. 2^n beats n^3 at $n = 10$ — exponentials really are hopeless immediately. But $n^{1.1}$ does not beat $n \lg n$ until $n \approx 5 \times 10^{17}$, which is more elements than you will ever have.* ***Both facts are consequences of the same hierarchy, and only one of them matters in practice.*** *This is the honest answer to "does asymptotic analysis lie?": it tells you the truth about the limit, and the limit is sometimes past the end of the world.*

Previous: *M01 — Foundations* · *Next:* *M03 — Divide & Conquer and Recurrences*